

Experiment 7: XOR Function Using Backpropagation with Momentum Factor

1. Title

Write a program for XOR function (binary input and output) with momentum factor using Backpropagation Algorithm.

2. Aim

To implement the XOR (Exclusive-OR) function using a Multilayer Perceptron (MLP) trained with the Backpropagation algorithm incorporating a momentum factor.

3. Objectives

1. To understand why XOR cannot be solved using a single-layer perceptron.
2. To implement a multilayer feed-forward neural network.
3. To understand the Backpropagation learning algorithm.
4. To implement the momentum term during weight updates.
5. To train a neural network for binary XOR input-output mapping.
6. To evaluate the trained network using the XOR truth table.

4. Theory

4.1 XOR Function

XOR stands for Exclusive-OR. It produces an output of **1 when the two inputs are different** and **0 when the two inputs are the same**.

The XOR truth table is:

X1	X2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

Therefore,

$$Y = X1 \oplus X2$$

4.2 Why XOR Cannot Be Solved by a Single-Layer Perceptron

The XOR problem is **not linearly separable**. A single-layer perceptron can create only one linear decision boundary, so it cannot correctly separate the XOR classes.

Therefore, a **multilayer neural network with a hidden layer** is required to solve XOR.

The network used in this experiment has:

2 Input Neurons → 2 Hidden Neurons → 1 Output Neuron

This architecture is specified in the laboratory handout.

5. Network Architecture

The neural network consists of:

- Input neurons = 2
- Hidden neurons = 2
- Output neurons = 1
- Hidden activation function = Sigmoid
- Output activation function = Sigmoid
- Learning rate = 0.5
- Momentum factor = 0.9
- Maximum epochs = 10,000
- Target MSE = 0.001

These parameters are given in the experiment handout.

6. Activation Function

The sigmoid activation function is used in both the hidden and output layers.

The sigmoid function is:

$$f(\text{net}) = 1 / (1 + e^{-(\text{net})})$$

Its derivative is:

$$f'(\text{net}) = f(\text{net})[1 - f(\text{net})]$$

The sigmoid function produces an output between 0 and 1, which is suitable for binary classification.

7. Backpropagation with Momentum

Backpropagation is a supervised learning algorithm used to train multilayer neural networks.

The network first performs **forward propagation** to calculate the output. The error between the target and predicted output is then calculated. This error is propagated backward through the network to calculate the weight corrections.

The momentum term uses the previous weight change along with the current gradient-based update.

The weight change is:

$$\Delta w(t) = -\eta(\partial E/\partial w) + \alpha \Delta w(t-1)$$

The new weight is:

$$w(t+1) = w(t) + \Delta w(t)$$

where:

- η = learning rate
- α = momentum factor
- $\Delta w(t-1)$ = previous weight change

Momentum helps the training process move faster and can reduce oscillations during learning.

8. Forward Propagation

For each hidden neuron:

$$net_j = \sum x_i w_{ij} + b_j$$

The hidden neuron output is:

$$h_j = f(net_j)$$

For the output neuron:

$$net_o = \sum h_j w_{jo} + b_o$$

The final network output is:

$$y = f(net_o)$$

Thus, the input is passed through the hidden layer and then to the output layer.

9. Error and Delta Calculations

The error is calculated as:

$$E = \frac{1}{2}(t - y)^2$$

The output-layer delta is:

$$\delta_o = (t - y)y(1 - y)$$

The hidden-layer delta is:

$$\delta_j = h_j(1 - h_j)\delta_{owjo}$$

These delta values are used to update the weights during backpropagation.

10. Weight Updates with Momentum

For the hidden-to-output layer:

$$\Delta w_{jo} = \eta \delta_o h_j + \alpha \Delta w_{jo}(\text{previous})$$

For the input-to-hidden layer:

$$\Delta w_{ij} = \eta \delta_j x_i + \alpha \Delta w_{ij}(\text{previous})$$

The updated weight is:

$$w(\text{new}) = w(\text{old}) + \Delta w$$

The momentum term therefore includes the previous weight change while calculating the current update.

11. Algorithm

1. Start.
2. Define the XOR training dataset.
3. Create a neural network with 2 input neurons, 2 hidden neurons and 1 output neuron.
4. Initialize weights and biases randomly.
5. Set the learning rate and momentum factor.
6. Initialize all previous weight changes to zero.
7. Perform forward propagation for each training pattern.
8. Calculate the output error.
9. Calculate the output and hidden-layer deltas.
10. Calculate weight and bias changes including the momentum term.
11. Update the weights and biases.
12. Calculate the Mean Squared Error (MSE).
13. Repeat the training until the target MSE is reached or the maximum number of epochs is completed.
14. Test the trained network using all four XOR patterns.
15. Display the predicted outputs and training-error graph.
16. Stop.

This follows the algorithm given in the laboratory handout.

12. Advantages

1. Solves the non-linearly separable XOR problem.
2. Momentum can accelerate convergence.
3. Momentum can reduce oscillations during training.
4. Demonstrates multilayer neural-network learning.
5. Provides an example of gradient-based optimization.

13. Applications

1. Pattern classification
2. Binary classification
3. Character recognition
4. Image recognition
5. Speech recognition
6. Decision-making systems
7. Nonlinear function approximation

14. Result

The XOR function with binary inputs and binary output was successfully implemented using a multilayer neural network trained with the Backpropagation algorithm and momentum factor.

The trained network learned the XOR input-output relationship and produced the expected binary outputs for the four XOR input combinations.

15. Conclusion

The XOR function was successfully implemented using a Multilayer Perceptron trained with the Backpropagation algorithm and momentum factor.

The experiment demonstrated that a multilayer neural network can solve the non-linearly separable XOR problem, which cannot be solved using a single-layer perceptron. The momentum factor improves the learning process by helping accelerate convergence and reducing oscillations during weight updates.

The trained network successfully produced the expected binary outputs for all XOR input combinations.